Project - Low Level Design on Dockerized Sports Python Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Team Members:

Sr no	Student Name	Enrolment Number
1	Priyanshi Gandhe	EN22CS301764
2	Priyanshu Sengar	EN22CS301770
3	Rudra Yadav	EN22CS301831
4	Prithviraj Singh Tomar	EN22CS301757
5	Pranjali Sharma	EN22CS301733

Table of Contents

1. **Introduction**
   1.1 Scope of the Document
   1.2 Intended Audience
   1.3 System Overview
2. **System Design**
   2.1 Application Design
      2.1.1 Architecture
      2.1.2 Layers Description
      2.1.3 Framework Used

   2.2 Process Flow

   2.3 Information Flow

   2.4 Components Design
      2.4.1 Streamlit Application
      2.4.2 SQLite Database
      2.4.3 Docker
      2.4.4 Docker Workflow

   2.5 Key Design Considerations
      2.5.1 Simplicity
      2.5.2 Portability
      2.5.3 Performance
      2.5.4 Security
      2.5.5 Reliability

   2.6 Feature Catalogue

3. **Deployment**
   3.1 Docker Setup
   3.2 Build and Run Process
   3.3 Application Access
4. **References**

1. Introduction

1.1 Scope of the Document

This document describes the low-level design of the **Dockerized Sports Python Service**, including application structure, APIs, and containerized deployment using Docker.

1.2 Intended Audience

- Developers
- DevOps Beginners
- Testers
- Project Evaluators / Faculty

1.3 System Overview

The system is a **lightweight sports management application** built using **Streamlit (Python)**. It allows users to manage sports-related data such as:

- Players
- Matches
- Scores

Key characteristics:

- Uses **SQLite** as a local database (no external DB service required)
- Fully containerized using **Docker**
- Runs as a **single-container application**

2. System Design

2.1 Application Design

Architecture

The system follows a **simplified layered architecture** suitable for small-scale applications.

Layers:

1. **UI Layer (Streamlit)**
    - Provides interactive frontend
    - Handles user inputs (forms, buttons)
    - Displays results
2. **Service Layer**
    - Contains business logic
    - Validates inputs

    - Processes user actions
3. **Data Access Layer**
    - Communicates with SQLite
    - Executes SQL queries
4. **Database Layer**
    - SQLite database file
    - Stores players, matches, and scores

Framework Used

- **Streamlit** (for UI + backend logic combined)

2.2 Process Flow

Step-by-step execution:

1. User interacts with Streamlit UI
2. Input is captured via forms/buttons
3. Validation is performed
4. Request is processed in service logic
5. SQLite query is executed
6. Data is retrieved/updated
7. Result is displayed on UI

2.3 Information Flow

User → Streamlit UI → Business Logic → SQLite Database → UI Response

Details:

- Data is handled internally (for api tavily and groq is used)
- SQLite stores data in file format
- Errors are handled and shown in UI messages

2.4 Components Design

1. Streamlit Application

- Main entry point (app.py)
- Handles:
    - UI rendering
    - Input handling
    - Logic execution

2. SQLite Database

- Lightweight, file-based database
- Stores:
  - Player details
  - Match records
  - Scores

3. Docker

- Used to containerize the application
- Includes:
  - Python environment
  - Required dependencies
  - Application code
- Key elements:
  - **Dockerfile**
  - **Docker Image**
  - **Docker Container**

5. Docker Workflow

1.Dockerfile

2.Build Docker Image

3.Run Docker Container

Command flow:

```
docker build -t streamlit-app .
docker run -p 8501:8501 streamlit-app
```

2.5 Key Design Considerations

1. Simplicity

- Single-container architecture
- Easy to understand and deploy

2. Portability

- Docker ensures app runs on any system

3. Performance

- SQLite is fast for small datasets

- Streamlit provides quick UI rendering

4. Security

- No sensitive data stored
- Environment kept isolated via Docker

5. Reliability

- Minimal dependencies reduce failure points

2.6 API / Feature Catalogue

Note: Since this is a **Streamlit-based app**, APIs are not exposed. Instead, features are handled via UI.

Features:

1. **View Matches**
    - Displays all match records
2. **Add Match**
    - Input teams
    - Stores in database
3. **View Players**
    - Fetch player details
4. **Add Player**
    - Insert new player data
5. **Update Score**
    - Modify match score

3. Deployment

Docker Deployment Steps

1.Create Dockerfile

```
# Use official Python image
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
```

```
    git \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements and install
COPY requirement.txt .
RUN pip install --no-cache-dir -r requirement.txt

# Copy the rest of the application
COPY . .

# Expose Streamlit port
EXPOSE 8501

# Command to run the application
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

2.Build image

3.Run container

Example:

docker build -t sports-service .
docker run -p 8501:8501 sports-service

App runs on:

http://localhost:8501

```
ubuntu@ip-172-31-15-158:~$ sudo su
root@ip-172-31-15-158:/home/ubuntu# sudo docker ps -a
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS                     PORTS     NAMES
5d2924bd3360   my-streamlit-app  "streamlit run app.p…"   18 hours ago   Exited (0) 18 hours ago              streamlit-container
0328cadc5cf4   15c5a26f7603      "/bin/sh -c 'apt-get…"   18 hours ago   Exited (100) 18 hours ago            flamboyant_satoshi
root@ip-172-31-15-158:/home/ubuntu# docker images
REPOSITORY        TAG        IMAGE ID       CREATED        SIZE
my-streamlit-app  latest     791e9d890787   18 hours ago   1.68GB
python            3.10-slim  f2d628de8a5f   27 hours ago   122MB
root@ip-172-31-15-158:/home/ubuntu#
```

4. References

- Python Official Documentation
- Streamlit Documentation
- SQLite Documentation
- Docker Documentation